



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

DATA ANALYSIS WITH PANDAS

Professor: *Gabriel E. Cabrera*
Teaching Assistant: *Alex Den Braber*



The Pandas library is designed for tabular data analysis. It not only provides useful classes and functions, it also allows you to apply functions from other libraries such as NumPy.

There are two fundamental structures in Pandas:

Table 1: Fundamental Structures

Object type	Description	Used for
DataFrame	2-dimensional object with an index (index)	Tabular data organized in columns
Series	1-dimensional object with an index (index)	A single (time) series of data

To import Pandas:

```
# pandas is imported
import pandas as pd
```

1 Series

A Series is a one-dimensional *array* that contains a sequence of values (as in NumPy) and a *label* called the index (**index**). To create a Series:

```
# a Series is created
obj = pd.Series([4, 7, -5, 3],          # the data is defined
                 index = ['a', 'b', 'c', 'd']) # the index is specified
```

```
# obj is checked
obj
```

```
## a    4
## b    7
## c   -5
## d    3
## dtype: int64
```

2 DataFrames

A DataFrame is a 2-dimensional structure with labeled data (*labels*), an index on the rows as well as on the columns. A column can potentially contain different data types. To create a **DataFrame** object:

```
# a DataFrame is created
df = pd.DataFrame([10, 20, 30, 40],          # the data is defined
                  columns = ['numbers'],      # column name
                  index = ['a', 'b', 'c', 'd']) # the index is specified

# df is checked
df
```

```
##      numbers
## a         10
## b         20
## c         30
## d         40
```

It is important to note that:

- The data is organized in columns (they can have custom names)
- There is an index that can take different formats (e.g. numbers, *strings*, etc.).

To access the index:

```
# extracts column
df.columns
```

```
## Index(['numbers'], dtype='object')
```

A Series can be passed to a DataFrame:

```
# series to df
obj_to_df = obj.to_frame()

# the column name is assigned to it
obj_to_df.columns = ['numbers']

# checks obj_to_df
obj_to_df
```

```
##      numbers
## a          4
## b          7
## c         -5
## d          3
```

In summary, the DataFrame structure:

The diagram illustrates the structure of a DataFrame. It shows a table with columns labeled 'Name', 'Team', 'Number', 'Position', 'Age', 'Height', 'Weight', 'College', and 'Salary'. The rows are indexed from 0 to 6. Annotations include: 'Column names' pointing to the column headers, 'Columns axis=1' pointing to the column headers, 'Index label' pointing to the row index, 'Index axis=0' pointing to the row index, 'Missing value' pointing to a 'NaN' value in the 'Number' column, and 'Data' pointing to the data values in the 'Weight' column. A small 'OG' logo is in the bottom right corner.

	Name	Team	Number	Position	Age	Height	Weight	College	Salary
0	Avery Bradley	Boston Celtics	0.0	PG	25.0	6-2	180.0	Texas	7730337.0
1	John Holland	Boston Celtics	30.0	SG	27.0	6-5	205.0	Boston University	NaN
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0	6-10	231.0	NaN	5000000.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0	6-8	235.0	LSU	1170960.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0	6-2	190.0	Louisville	1824360.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN	6-9	260.0	Ohio State	2569260.0
6	Evan Turner	Boston Celtics	11.0	SG	27.0	6-7	220.0	Ohio State	3425510.0

To see the documentation of the Pandas library go to: <https://pandas.pydata.org/docs/>

3 Application

1. The files `gapminder.xlsx` and `gapminder.dta` contain an extract from the Gapminder project on life expectancy (`lifeExp`), GDP per capita (`gdpPercap`) and population (`pop`), by country (`continent`). Using the Pandas library, load both datasets into your workspace. Name one of the two `DataFrame` objects `gapminder`.
2. Show the first and last 10 observations of `gapminder`.
3. Generate a new `DataFrame` that contains only the countries of the American continent (`americas`) in the year 2007.
4. Based on the `DataFrame` generated in (3), show:
 - The country with the highest GDP per capita in the year 2007
 - The country with the lowest GDP per capita in the year 2007

What do you notice about the index?

5. Reset the index of the `DataFrame` generated in (3). Then drop the variables `country` and `continent`.
6. Using a *list comprehension*, rename the columns with their original name in lowercase.
7. Generate a variable that contains life expectancy (`lifeexp`) in months. Do it by defining a function and by using an anonymous function.
8. Using the original dataset:
 - a. Work only with the European countries.
 - b. Generate the growth of GDP per capita by country.
 - c. Drop the `NAs`.
 - d. Build a brief descriptive statistics summary by country.
 - e. Save the descriptive statistics in a file with the `.xlsx` (excel) extension.